

ECS 409/609

Assignment 6

Sequential Construct-II

Floating Point Instructions, Input Output Instructions and Comparison Instructions

Kunwar Arpit Singh

Submission Deadline: 26 October, 2025

For code repository of this assignment:  **MIPS Assignments**

1. Problem 1

Write an MIPS assembly program that takes user's name as an input and prompts the message

Hi Ajay, MIPS assembly programming is very exciting to learn

MIPS Assembly Source

```
1 # Assignment 6: Sequential Construct-II
2 # Programmed By : Kunwar Arpit Singh
3
4 .data
5     prompt_msg: .asciiz "Programmed By: Kunwar Arpit Singh 22185\n\nPlease
6         enter your first name:"
7     greeting_start: .asciiz "Hi "
8     greeting_end: .asciiz ", MIPS assembly programming is very exciting to
9         learn"
10    user_name: .space 20
11
12 .text
13 .globl main
14
15 main:
16     li $v0, 4
17     la $a0, prompt_msg
18     syscall
19
20     li $v0, 8
21     la $a0, user_name
22     li $a1, 20
23     syscall
24
25     la $t0, user_name
26
27 find_newline:
```

```

26     lb $t1, 0($t0)
27
28     beq $t1, 10, replace_newline
29     beq $t1, 0, exit_loop
30
31     addi $t0, $t0, 1
32     j find_newline
33
34 replace_newline:
35     sb $zero, 0($t0)
36
37 exit_loop:
38
39     li $v0, 4
40     la $a0, greeting_start
41     syscall
42
43     li $v0, 4
44     la $a0, user_name
45     syscall
46
47     li $v0, 4
48     la $a0, greeting_end
49     syscall
50
51     li $v0, 10
52     syscall

```

Output Screenshots

Register values and Console

The screenshot displays a debugger interface with two main panels. The left panel, titled 'Int Regs [10]', shows the current state of various registers. The right panel, titled 'Console', shows the output of the program.

Register Values (Int Regs [10]):

- PC = 4194460
- EPC = 0
- Cause = 0
- BadVAddr = 0
- Status = 805371664
- HI = 0
- LO = 0
- R0 [r0] = 0
- R1 [at] = 268500992
- R2 [v0] = 10
- R3 [v1] = 0
- R4 [a0] = 268501068
- R5 [a1] = 20
- R6 [a2] = 2147479336
- R7 [a3] = 0
- R8 [t0] = 268501128
- R9 [t1] = 10
- R10 [t2] = 0
- R11 [t3] = 0
- R12 [t4] = 0
- R13 [t5] = 0
- R14 [t6] = 0
- R15 [t7] = 0
- R16 [s0] = 0
- R17 [s1] = 0
- R18 [s2] = 0
- R19 [s3] = 0
- R20 [s4] = 0
- R21 [s5] = 0
- R22 [s6] = 0
- R23 [s7] = 0
- R24 [t8] = 0
- R25 [t9] = 0
- R26 [k0] = 0
- R27 [k1] = 0
- R28 [gp] = 268468224
- R29 [sp] = 2147479312
- R30 [s8] = 0
- R31 [ra] = 4194328

Console Output:

```

Programmed By: Runwar Arpit Singh 22185
Please enter your first name: Kartik
Hi Kartik, MIPS assembly programming is very exciting to learn

```

Kernel Text Segment [80000000]..[80010000]:

```

[80000180] 0001d821 addu $27, $0, $1 ; 90: move $k1 $at # Save $at
[80000184] 3c019000 lui $1, -28672 ; 92: sw $v0 $1 # Not re-entrant and we can't trust $sp
[80000188] ac220200 sw $2, 512($1) ; 93: sw $a0 $2 # But we need to use these registers
[8000018c] 3c019000 lui $1, -28672 ; 95: mfc0 $k0 $13 # Cause register
[80000190] ac240204 sw $4, 516($1) ; 96: srl $a0 $k0 2 # Extract ExCoDe Field
[80000194] 401a6800 mfc0 $26, $13
[80000198] 001a2082 srl $4, $26, 2

```

2. Problem 2

Write an assembly program that takes principle amount, rate of interest and time (taken as an input to integer register) as an input from the user and calculate simple interest and display the raw and absolute results to the user.

MIPS Assembly Source

```
1 # Assignment 6: Sequential Construct-II
2 # Programmed By : Kunwar Arpit Singh
3 .data
4     promptP: .asciiz "Programmed By: Kunwar Arpit Singh\n\nEnter Principle (P)
5                 _as an integer:_"
6     promptR: .asciiz "Enter Rate (R) as an integer:_"
7     promptT: .asciiz "Enter Time (T) as an integer:_"
8     msgSI: .asciiz "\nSimple Interest (SI):_"
9     msgAbs: .asciiz "\nAbsolute SI:_"
10 .text
11 .globl main
12 main:
13     li $v0, 4
14     la $a0, promptP
15     syscall
16     li $v0, 5
17     syscall
18     move $t0, $v0
19
20     li $v0, 4
21     la $a0, promptR
22     syscall
23     li $v0, 5
24     syscall
25     move $t1, $v0
26
27     li $v0, 4
28     la $a0, promptT
29     syscall
30     li $v0, 5
31     syscall
32     move $t2, $v0
33
34     mtc1 $t0, $f1
35     cvt.s.w $f1, $f1
36     mtc1 $t1, $f2
37     cvt.s.w $f2, $f2
38     mtc1 $t2, $f3
39     cvt.s.w $f3, $f3
40     li.s $f10, 100.0
41     mul.s $f4, $f1, $f2
42     mul.s $f4, $f4, $f3
43     div.s $f12, $f4, $f10
```

```

43
44     li $v0, 4
45     la $a0, msgSI
46     syscall
47
48     li $v0, 2
49     syscall
50
51     abs.s $f12, $f12
52
53     li $v0, 4
54     la $a0, msgAbs
55     syscall
56
57     li $v0, 2
58     syscall
59
60     li $v0, 10
61     syscall

```

Output Screenshots

Register values and Console

The screenshot displays a debugger window with two panes. The left pane, titled 'Int Regs [10]', shows the current state of MIPS registers. The right pane shows a 'Console' window with the program's output and a list of instructions with their corresponding register values.

Register Values:

Register	Value
PC	4194520
EPC	0
Cause	0
BadVAddr	0
Status	805371664
HI	0
LO	0
R0 [r0]	0
R1 [at]	268500992
R2 [v0]	10
R3 [v1]	0
R4 [a0]	268501150
R5 [a1]	2147479316
R6 [a2]	2147479336
R7 [a3]	0
R8 [t0]	100
R9 [t1]	50
R10 [t2]	5
R11 [t3]	0
R12 [t4]	0
R13 [t5]	0
R14 [t6]	0
R15 [t7]	0
R16 [s0]	0
R17 [s1]	0
R18 [s2]	0
R19 [s3]	0
R20 [s4]	0
R21 [s5]	0
R22 [s6]	0
R23 [s7]	0
R24 [t8]	0
R25 [t9]	0
R26 [k0]	0
R27 [k1]	0
R28 [gp]	268468224
R29 [sp]	2147479312
R30 [s8]	0
R31 [ra]	4194328

Console Output:

```

Programmed By: Kunwar Arpit Singh

Enter Principle (P) as an integer: 100
Enter Rate (R) as an integer: 50
Enter Time (T) as an integer: 5

Simple Interest (SI): 250.00000000
Absolute SI: 250.00000000

```

Instruction Log:

Address	Instruction	Comment
[004000b8]	46006305 abs.s \$f12, \$f12	; 62: abs.s \$f12, \$f12
[004000bc]	34020004 ori \$2, \$0, 4	; 64: li \$v0, 4
[004000c0]	3c011001 lui \$1, 4097 [msgAbs]	; 65: la \$a0, msgAbs
[004000c4]	3424009e ori \$4, \$1, 158 [msgAbs]	
[004000c8]	0000000c syscall	; 66: syscall
[004000cc]	34020002 ori \$2, \$0, 2	; 68: li \$v0, 2
[004000d0]	0000000c syscall	; 69: syscall
[004000d4]	3402000a ori \$2, \$0, 10	; 71: li \$v0, 10
[004000d8]	0000000c syscall	; 72: syscall

3. Problem 3

Problem statement

Write an assembly program that takes three character string from the user and print the second character from it. (Hint: use lbu instruction).

MIPS Assembly Source

```
1 # Assignment 6: Sequential Construct-II
2 # Problem 3
3 # Programmed By : Kunwar Arpit Singh
4
5 .data
6     prompt: .asciiz "Programmed By: Kunwar Arpit Singh\nEnter a 3-character
7               string:"
8     message: .asciiz "\nThe 2nd char in the string:"
9     buffer: .space 5
10
11 .text
12 .globl main
13 main:
14     li $v0, 4
15     la $a0, prompt
16     syscall
17
18     li $v0, 8
19     la $a0, buffer
20     li $a1, 5
21     syscall
22
23     li $v0, 4
24     la $a0, message
25     syscall
26
27     la $t0, buffer
28     lbu $a0, 1($t0)
29
30     li $v0, 11
31     syscall
32
33     li $v0, 10
34     syscall
```

Output Screenshots

Register values and Console

The screenshot displays a debugger interface with two main panels. The left panel, titled 'Int Regs [10]', shows the current state of various MIPS registers. The right panel, titled 'Console', shows the output of a program.

Register Values (Int Regs [10]):

- PC = 4194412
- EPC = 0
- Cause = 0
- BadVAddr = 0
- Status = 805371664
- HI = 0
- LO = 0
- R0 [r0] = 0
- R1 [at] = 268500992
- R2 [v0] = 10
- R3 [v1] = 0
- R4 [a0] = 116
- R5 [a1] = 5
- R6 [a2] = 2147479336
- R7 [a3] = 0
- R8 [t0] = 268501085
- R9 [t1] = 0
- R10 [t2] = 0
- R11 [t3] = 0
- R12 [t4] = 0
- R13 [t5] = 0
- R14 [t6] = 0
- R15 [t7] = 0
- R16 [s0] = 0
- R17 [s1] = 0
- R18 [s2] = 0
- R19 [s3] = 0
- R20 [s4] = 0
- R21 [s5] = 0
- R22 [s6] = 0
- R23 [s7] = 0
- R24 [t8] = 0
- R25 [t9] = 0
- R26 [k0] = 0
- R27 [k1] = 0
- R28 [gp] = 268468224
- R29 [sp] = 2147479312
- R30 [s8] = 0
- R31 [ra] = 4194328

Console Output:

```
Programmed By: Kunwar Arpit Singh
Enter a 3-Character string: str
The 2nd char in the string: t
```

Assembly Code Snippet:

```
[80000188] ac220200 sw $2, 512($1)
[8000018c] 3c019000 lui $1, -28672 ; 93: sw $a0 $2 # But we need to use these registers
[80000190] ac240204 sw $4, 516($1)
[80000194] 401a6800 mfc0 $26, $13 ; 95: mfc0 $k0 $13 # Cause register
[80000198] 001a2082 srl $4, $26, 2 ; 96: srl $a0 $k0 2 # Extract ExCoDe Field
[8000019c] 3084001f andi $4, $4, 31 ; 97: andi $a0 $a0 0x1f
[800001a0] 34020004 ori $2, $0, 4 ; 101: li $v0 4 # syscall 4 (print_str)
[800001a4] 3c049000 lui $4, -28672 (__m1_) ; 102: la $a0 __m1_
[800001a8] 0000000c syscall ; 103: syscall
```

4. Problem 4

Problem statement

Write an assembly program that takes two strings (of length of two characters) from the user and calculate the hamming distance. For example if hi and he are the input from the user then the hamming distance is 01.

MIPS Assembly Source

```
1 # Assignment 1: Sequential Construct-I
2 # Programmed By : Kunwar Arpit Singh
3
4 .data
5     prompt1: .asciiz "Programmed By: Kunwar Arpit Singh 22185\n\nEnter first 2-
6         char string: "
7     prompt2: .asciiz "\nEnter second 2-char string: "
8     result: .asciiz "\nHamming distance: "
9
10    buffer1: .space 3
11    buffer2: .space 3
12
13 .text
14 .globl main
15 main:
16
17     li $t0, 0
18
19     li $v0, 4
20     la $a0, prompt1
21     syscall
22     li $v0, 8
23     la $a0, buffer1
24     li $a1, 3
25     syscall
26
27     li $v0, 4
28     la $a0, prompt2
29     syscall
30
31     li $v0, 8
32     la $a0, buffer2
33     li $a1, 3
34     syscall
35
36     la $t1, buffer1
37     la $t2, buffer2
38
39     lbu $t3, 0($t1)
40     lbu $t4, 0($t2)
```

```

41
42     sne $t5, $t3, $t4
43     add $t0, $t0, $t5
44
45     lbu $t3, 1($t1)
46     lbu $t4, 1($t2)
47
48     sne $t5, $t3, $t4
49     add $t0, $t0, $t5
50
51     li $v0, 4
52     la $a0, result
53     syscall
54
55     li $v0, 1
56     move $a0, $t0
57     syscall
58
59     li $v0, 10
60     syscall

```

Output Screenshots

Register values and Console

The screenshot displays a debugger interface with two main panels. The left panel, titled 'Int Regs [10]', shows the current state of MIPS registers. The right panel, titled 'Console', shows the program's output.

Register Values (Int Regs [10]):

- PC = 4194524
- EPC = 0
- Cause = 0
- BadVAddr = 0
- Status = 805371664
- HI = 0
- LO = 0
- R0 [r0] = 0
- R1 [at] = 268500992
- R2 [v0] = 10
- R3 [v1] = 0
- R4 [a0] = 2
- R5 [a1] = 3
- R6 [a2] = 2147479336
- R7 [a3] = 0
- R8 [t0] = 2
- R9 [t1] = 268501111
- R10 [t2] = 268501114
- R11 [t3] = 116
- R12 [t4] = 52
- R13 [t5] = 1
- R14 [t6] = 0
- R15 [t7] = 0
- R16 [s0] = 0
- R17 [s1] = 0
- R18 [s2] = 0
- R19 [s3] = 0
- R20 [s4] = 0
- R21 [s5] = 0
- R22 [s6] = 0
- R23 [s7] = 0
- R24 [t8] = 0
- R25 [t9] = 0
- R26 [k0] = 0
- R27 [k1] = 0
- R28 [gp] = 268468224
- R29 [sp] = 2147479312
- R30 [s8] = 0
- R31 [ra] = 4194328

Console Output:

```

Programmed By: Kunwar Arpit Singh 22185

Enter first 2-char string: 4t
Enter second 2-char string: 54
Hamming distance: 2

```

Disassembly (Text Panel):

```

[004000bc] 34020004 ori $2, $0, 4           ; 51: li $v0, 4
[004000c0] 3c011001 lui $1, 4097 [result] ; 52: la $a0, result
[004000c4] 34240063 ori $4, $1, 99 [result]
[004000c8] 0000000c syscall          ; 53: syscall
[004000cc] 34020001 ori $2, $0, 1           ; 55: li $v0, 1
[004000d0] 00082021 addu $4, $0, $8      ; 56: move $a0, $t0
[004000d4] 0000000c syscall          ; 57: syscall
[004000d8] 3402000a ori $2, $0, 10          ; 59: li $v0, 10
[004000dc] 0000000c syscall          ; 60: syscall

```